

ARTIFICIAL INTELLIGENCE, BLOCKCHAIN E CRIPTOVALUTE NELLO SVILUPPO SOFTWARE

Manuale Universitario Integrato

Basato sulle lezioni del Prof. Giancarlo Succi – Università di Bologna

ARTIFICIAL INTELLIGENCE, BLOCKCHAIN E CRIPTOVALUTE NELLO SVILUPPO SOFTWARE

Manuale Universitario Integrato – Primavera 2026

Basato sulle lezioni del Prof. Giancarlo Succi

INTRODUZIONE AL CORSO E FILOSOFIA

Il corso adotta una modalità ibrida sperimentale per favorire gli studenti lavoratori.

- **Obiettivo:** Costruire modelli di produzione software complessi integrando AI, Blockchain e modelli cognitivi ("Learning to see").
 - **Strumenti:** Si utilizza la piattaforma di Constructor Group (LLMs, reti semantiche e HPC). La documentazione va prodotta in LaTeX.
 - **Filosofia:** Il docente raccomanda l'uso di carta e penna per il ragionamento logico prima della codifica. Severità assoluta sul plagio.
-

PARTE I – FONDAMENTI TEORICI ED ECONOMICI

Capitolo 1: La natura economica del software

Il software è il risultato di un atto creativo protetto dal diritto d'autore. I diritti si dividono in Morali (inalienabili, es. paternità) ed Economici (alienabili/cedibili in licenza d'uso).

- **Funzione di Costo a "L":**
 - $C(n) = C_f + (n \times \epsilon)$
 - $C(n) = C$
 - f
 - $+ (n \times \epsilon)$
 - . Alti costi fissi iniziali per lo sviluppo (C_f)
 - C
 - f
 - ϵ , costi marginali (ϵ)
 - ϵ
 - ϵ
 -) quasi nulli per la replicazione.
- **Esternalità di Rete e Metcalfe:** Il valore del software aumenta in modo quadratico rispetto al numero di utenti ($Valore = n^2$)
 - $Valore = n$
 - 2
 -).
- **Effetti sul mercato:** Generano "Insufficient friction" (tutti adottano la tecnologia velocemente) e posizioni di monopolio. Le aziende sfruttano il **Lock-in** (barriere all'uscita) e il **Bundling** (pacchettizzazione di più software, es. Microsoft Office).

Capitolo 2: Le radici industriali (Taylorismo e Fordismo)

- **Taylorismo (Scientific Management):** Si basa sulla scomposizione di compiti complessi in sottoproblemi semplici e sulla separazione tra chi pensa (manager) e chi esegue (operaio). Nel software ha ispirato la logica della pianificazione rigida.
- **Fordismo ed Economie di Scala:** Mira ad abbattere i costi medi producendo in lotti.
 - Formula della produzione isolata:

Formula della produzione isolata: $t_a = p + a + w + d$

Formula in lotto: $T_b = p + a + n \times w + d$

- **Il Paradosso:** Nel software, il costo di lavorazione della replica è zero. Le economie di scala sono massime, ma la rigidità fordista/taylorista fallisce di fronte all'incertezza intrinseca del software.

Capitolo 3: La Software Crisis e le difficoltà di Brooks

Fenomeno definito da Dijkstra (1972): la potenza hardware cresce più velocemente della nostra capacità di gestire la programmazione.

Le 4 difficoltà strutturali (Fred Brooks):

1. **Complessità:** Il software ha più stati possibili e intrecciati di qualsiasi sistema fisico.
2. **Conformità:** Deve adattarsi a interfacce, istituzioni e leggi preesistenti (spesso irrazionali).
3. **Modificabilità:** Il software "vivo" viene costantemente cambiato e aggiornato.
4. **Invisibilità:** Essendo immateriale, è difficile da visualizzare e comunicare.

Capitolo 4: Teoria dei Sistemi e Regimi di Mensch

Lo sviluppo software è un sistema dinamico. Può trovarsi in diversi "regimi":

1. **Pseudo-deterministico (PDS):** Sembra stabile in condizioni normali, ma nasconde fragilità improvvise ("Illusione del tacchino" o "Illusione della prevedibilità").
2. **Caotico Deterministico (DCS):** Governato da equazioni precise, ma una variazione infinitesima nelle condizioni iniziali porta a risultati imprevedibili (Effetto Farfalla).

3. **Stocastico (StS):** C'è incertezza, ma mantiene una struttura probabilistica stabile (permette previsioni statistiche e gestione del rischio).
4. **Randomicità Stocastica (SR):** Caos puro. La previsione è impossibile, si può solo fare contenimento danni.

Nota bene: Confondere il regime di un sistema (pensare che sia PDS quando in realtà è Caotico/Stocastico) è la causa principale dei fallimenti nel software.

Capitolo 5: Problemi Tame e Wicked

- **Tame (Mansueti):** Problemi ben definiti, con una soluzione "giusta" o "sbagliata", appartenenti a classi simili (approccio Waterfall/Taylorista).
- **Wicked (Perversi):** Non hanno una formulazione definitiva. Ogni tentativo di soluzione cambia il problema stesso. Le soluzioni non sono "vere/false" ma "accettabili/non accettabili". Il software è un problema Wicked.

Capitolo 6: Meccanismi di Coordinamento (Malone & Crowston)

Come si coordinano le persone per produrre software:

1. **Sequenziale:** Il deliverable passa da un reparto all'altro. Facile ma rigido.
2. **Risorse Condivise:** Tutti pescano da una coda di task comune. Richiede priorità chiare.
3. **Risultato Comune (Sincrono):** Il team lavora insieme contemporaneamente sullo stesso output (es. Pair Programming). È il più difficile perché richiede unanimità, ma indispensabile per i problemi Wicked.

PARTE II – MODELLI COGNITIVI E PERCEZIONE

Capitolo 7: L'Epistemologia di Gregory Bateson

La base per capire l'Intelligenza Artificiale è capire come l'uomo percepisce:

- **Informazione:** "Una differenza che produce una differenza".
- **Abduzione:** Ragionamento basato sulla ricerca e comparazione di pattern.

- **Pleroma vs Creatura:**
 - *Pleroma: Mondo non vivente, misurabile con forza e urti.*
 - *Creatura: Mondo dei sistemi viventi, mosso dalle differenze e dall'informazione.*
-
- **Mito del Potere:** *L'errore di voler gestire la "Creatura" (team di esseri umani) usando logiche del "Pleroma" (forza e controllo rigido).*

Capitolo 8: Patologie della Comunicazione e Apprendimento

Nei team si possono creare dinamiche distruttive:

- **Schismogenesi:** *Escalation di conflitto (Simmetrica = scontro alla pari; Complementare = dominio/sottomissione).*
- **Doppio Legame (Double Bind):** *Paradosso in cui si ricevono messaggi contraddittori su più livelli (es. "Lavora in autonomia ma segui le regole alla lettera"), punendo chi non li soddisfa entrambi.*
- **Deuterolearning:** *"Imparare a imparare". Il processo con cui le organizzazioni sane riflettono sui propri errori passati (metacognizione).*

Capitolo 9: Memoria, Razionalità ed Euristiche

- **La Memoria:** *Si divide in Registri sensoriali, Memoria a Breve Termine (STM, capacità limitata a*
- *7±2*
- *7±2*
- *elementi) e Memoria a Lungo Termine (LTM, Esplicita e Implicita). C'è inoltre la Working Memory usata per il calcolo attivo.*
- **Evoluzione del Ragionamento:**
 - *Naïve Scientist: L'uomo cerca cause logiche per tutto (genera Attribution Error).*
 - *Cognitive Miser (Avaro Cognitivo): L'uomo risparmia energia mentale usando **Euristiche** (es. ancoraggio, disponibilità) e stereotipi. Spesso veloci, ma a volte errate.*
 - *Motivated Tactician: L'individuo sceglie la strategia mentale adatta al contesto (riflessiva o impulsiva).*

-

Capitolo 10: Cognizione Distribuita e Pensiero Sistemico

- **Mente Estesa (Clark e Chalmers):** L'ambiente e gli strumenti (PC, carta, IDE) fanno fisicamente parte del processo cognitivo. La mente "esce" dal cervello (es. il taccuino di Otto malato di Alzheimer diventa la sua memoria).
 - **Vygotsky e ZPD:** L'apprendimento è sociale. La **Zona di Sviluppo Prossimale (ZPD)** è lo spazio tra ciò che so fare e ciò che potrei fare con l'aiuto di un mentore (**Scaffolding**). Il Pair Programming si basa su questo.
 - **Pensiero Sistemico (Weinberg):** Studia il software considerando le interdipendenze tra le persone (Homeostasis, resistenza al cambiamento). Lo strumento d'indagine principe è l'**Etnografia Cognitiva** (lo scienziato si immerge nel team).
 - **Il rischio del Conformismo:** Negli studi di Asch e Milgram si nota come la pressione sociale o l'obbedienza all'autorità possano spingere sviluppatori intelligenti ad accettare soluzioni errate o a compiere atti non etici.
-

PARTE III – ARTIFICIAL INTELLIGENCE E NEUROSCIENZE

Capitolo 11: L'Intelligenza Artificiale nello Sviluppo

Il corso vede l'AI applicata allo sviluppo in due modi:

1. **Modeling (Modellazione):** Usare la tecnologia per capire come la mente del programmatore lavora.
2. **Emulating (Emulazione):** Algoritmi che replicano le azioni umane (Machine Learning, Data Science).

Capitolo 12: Misurazione dei Segnali Cerebrali (EEG)

L'EEG (Elettroencefalografia) è lo strumento d'elezione per studiare i programmatori: è portatile, economico e ha un'altissima risoluzione temporale rispetto alla fMRI. Usa elettrodi posizionati secondo lo schema "10-20".

Onde Cerebrali:

- Delta (1-5 Hz): Sonno e inconscio.
- Theta (4-8 Hz): Carico cognitivo e memoria.
- **Alpha (8-12 Hz):** Rilassamento vigile. Se diminuisce, indica sforzo.
- **Mu (**
- μ
- μ
- **) (7.5-12.5 Hz):** Attività motorie e neuroni specchio.
- Beta (12-25 Hz): Concentrazione attiva.

Capitolo 13: Pipeline Tecnica con MNE-Python

Come si analizzano tecnicamente i dati EEG:

1. `read_raw_edf`: Lettura dei dati.
2. `notch_filter`: Filtro per rimuovere il rumore dei cavi elettrici europei (50 Hz).
3. **Calcolo IAF (Individual Alpha Frequency):** L'Alpha diminuisce con l'età, quindi la

frequenza viene calibrata individualmente:

$$IAF = 11.95 - (0.053 \times Et\grave{a})$$

4. **Calcolo ERD/ERS:** Si calcola l'energia del segnale.

$$ERD = \frac{Ampiezza_{Rest} - Ampiezza_{Programming}}{Ampiezza_{Programming}} \times 100\%$$

- **ERD positivo (Desincronizzazione):** Il cervello fa fatica, è in problem solving.
- **ERS negativo (Sincronizzazione):** Il cervello è a riposo.

PARTE IV – BLOCKCHAIN E METODOLOGIE AGILI

Capitolo 14: Blockchain e Decentralizzazione

La Blockchain è un sistema decentralizzato a retroazione negativa che mira alla stabilità e immutabilità del registro, eliminando il singolo punto di fallimento ("Single point of failure").

Struttura dati di un blocco (Esempio Python):

```
blocco = {  
    'indice': 1,  
    'timestamp': '2026-03-28 10:00:00',  
    'dati': 'Esecuzione Smart Contract',  
    'hash_precedente': '0000abc123...',  
    'nonce': 45632,  
    'hash_attuale': '0000def456...'  
}
```

L'Intelligenza Artificiale si integra qui analizzando (Modeling) gli **Smart Contract** per individuare vulnerabilità prima del rilascio, spostando l'incertezza da uno stato "Casuale" a "Stocastico".

Capitolo 15: Lean, Agile e XP (Extreme Programming)

Di fronte a problemi "Wicked" e regimi "Caotici/Stocastici", i modelli tradizionali (Waterfall) falliscono. Nasce il bisogno dell'Agile.

I 7 Sprechi (Muda) del Lean Software Development:

1. **Lavoro parziale:** Codice scritto ma non testato/integrato (è un costo, non un asset).
2. **Processi extra:** Burocrazia inutile.
3. **Funzionalità extra (Overproduction):** Sviluppare "nel caso serva" (il 64% delle feature non viene mai usato).
4. **Task Switching:** Perdita di contesto cambiando troppi progetti.
5. **Attese:** Colli di bottiglia nei test o nelle firme.
6. **Movimento:** Passaggi informativi inefficaci.
7. **Difetti:** Scoperti in ritardo, costano troppo.

Extreme Programming (XP):

- Valori: Semplicità, Comunicazione, Feedback, Coraggio.
 - Principi: Focus sul Valore, Flusso costante, Assenza di difetti.
 - Pratiche: Pair Programming, Integrazione Continua, TDD, Refactoring, Cliente sul posto.
-

APPENDICE: PREPARAZIONE ESAME (Focus del Docente)

1. **Matrice di Harris per il controllo del processo:**
 - Outcome: Misuro il risultato (vendite).
 - Behavior: Misuro i passaggi, conosco il processo alla perfezione (Taylorismo).
 - Clan/Group: Nessuna delle precedenti, serve fiducia e team unito (Agile, Problemi Wicked).
2. **Ariane 5 (1996):** È l'esempio classico del fallimento del riutilizzo di software (da Ariane 4) su sistemi fisici diversi. Un errore di overflow (inserimento a 64-bit in uno spazio a 16-bit) causò danni per 8 miliardi di dollari.
3. **Atteggimento all'Esame:** Leggere le dinamiche di ingegneria non solo come "codice", ma come interazione umana, psicologica (Bateson, Vygotsky) ed economica.

15 April

a slide che hai condiviso illustra i **5 Assiomi della Comunicazione** formulati da Paul Watzlawick e dalla Scuola di Palo Alto. Sono concetti fondamentali della psicologia e della sociologia.

Ecco degli **esempi pratici e quotidiani** per comprendere ciascuno dei 5 assiomi:

1. "One cannot not communicate" (Non si può non comunicare)

Anche se cerchi di non trasmettere messaggi, il tuo comportamento comunicherà qualcosa.

- **Esempio pratico:** Sei in treno. Ti siedi, metti le cuffie, apri un libro e non guardi in faccia nessuno. Non stai dicendo una parola, ma **stai comunicando benissimo** agli altri passeggeri: "Non voglio essere disturbato, non voglio parlare". Il silenzio e l'inattività sono a tutti gli effetti comunicazione.

2. "Every communication has a content and relationship aspect..." (Ogni comunicazione ha un aspetto di contenuto e uno di relazione)

Il "contenuto" è COSA dici, la "relazione" è COME lo dici e definisce il rapporto tra chi parla.

- **Esempio pratico:** Prendi la frase: "Chiudi la porta".
 - Se te lo dice un collega con un sorriso e tono pacato, il messaggio è una semplice **richiesta** di collaborazione (relazione alla pari).
 - Se te lo urla il tuo capo sbattendo i pugni sul tavolo, il messaggio è un **ordine** che rimarca la sua autorità (relazione di potere).Il contenuto (l'azione di chiudere la porta) è identico, ma la relazione ("metacomunicazione") cambia completamente il significato del messaggio.

●

3. "The nature of a relationship is dependent on the punctuation..." (La natura di una relazione dipende dalla punteggiatura delle sequenze di comunicazione)

Le persone interpretano cause ed effetti dei comportamenti in modo diverso, mettendo il "punto di inizio" della discussione in momenti differenti.

- **Esempio pratico:** Il classico litigio di coppia.
 - Lei dice: "Io brontolo di continuo perché tu ti chiudi in te stesso e mi ignori!" (Punteggiatura di lei: l'isolamento di lui è la **causa**, il brontolare è l'effetto).
 - Lui risponde: "Io mi chiudo in me stesso perché tu brontoli di continuo!" (Punteggiatura di lui: il brontolare di lei è la **causa**, il suo isolamento è l'effetto). Entrambi sono convinti di star solo reagendo al comportamento dell'altro.

●

4. "Human communication involves both digital and analog modalities" **(La comunicazione umana coinvolge modalità numeriche/digitali e analogiche)**

Il modulo "digitale" sono le parole usate. Il modulo "analogico" è il linguaggio non verbale (tono della voce, postura, espressioni).

- **Esempio pratico:** Hai appena fatto un regalo a un amico. Lui lo scarta, fa un sorriso tirato, abbassa lo sguardo, parla con voce piatta e dice: "Oh, grazie... mi piace tantissimo".
 - **Digitale (parole):** Dice che gli piace.
 - **Analogico (non verbale):** Il tono e la postura dicono che è deluso.
Spesso, quando i due moduli sono in contrasto, noi crediamo a quello analogico (il corpo non mente).
-

5. "Inter-human communication procedures are either symmetric or complementary" **(Gli scambi comunicativi sono simmetrici o complementari)**

Le relazioni si basano sull'uguaglianza (simmetria) o sulla differenza di ruolo (complementarità).

- **Esempio pratico (Simmetrico):** Due fratelli o due colleghi dello stesso livello discutono su dove andare a cena. Uno alza la voce per imporre la sua idea, l'altro alza la voce a sua volta. Si pongono sullo **stesso piano** (alla pari).
- **Esempio pratico (Complementare):** Un medico e un paziente. Il medico prescrive una cura (ruolo dominante/esperto, detto one-up), il paziente ascolta e accetta la terapia (ruolo sottomesso/recettivo, detto one-down). La comunicazione funziona perché c'è una **differenza di ruoli** accettata da entrambi.